

LearnRAG — Research Assistant

Project Documentation

Contents

LearnRAG — Research Assistant	1
What This Project Does	1
Pipeline Overview	2
Project Structure	2
Module Reference	3
Setup	4
Why Hybrid Search + Reranking (Not Just Dense Retrieval)	5
Evaluation Harness	5
Known Limitations / Notes	6
Possible Next Steps	6
Tech Stack	6

LearnRAG — Research Assistant

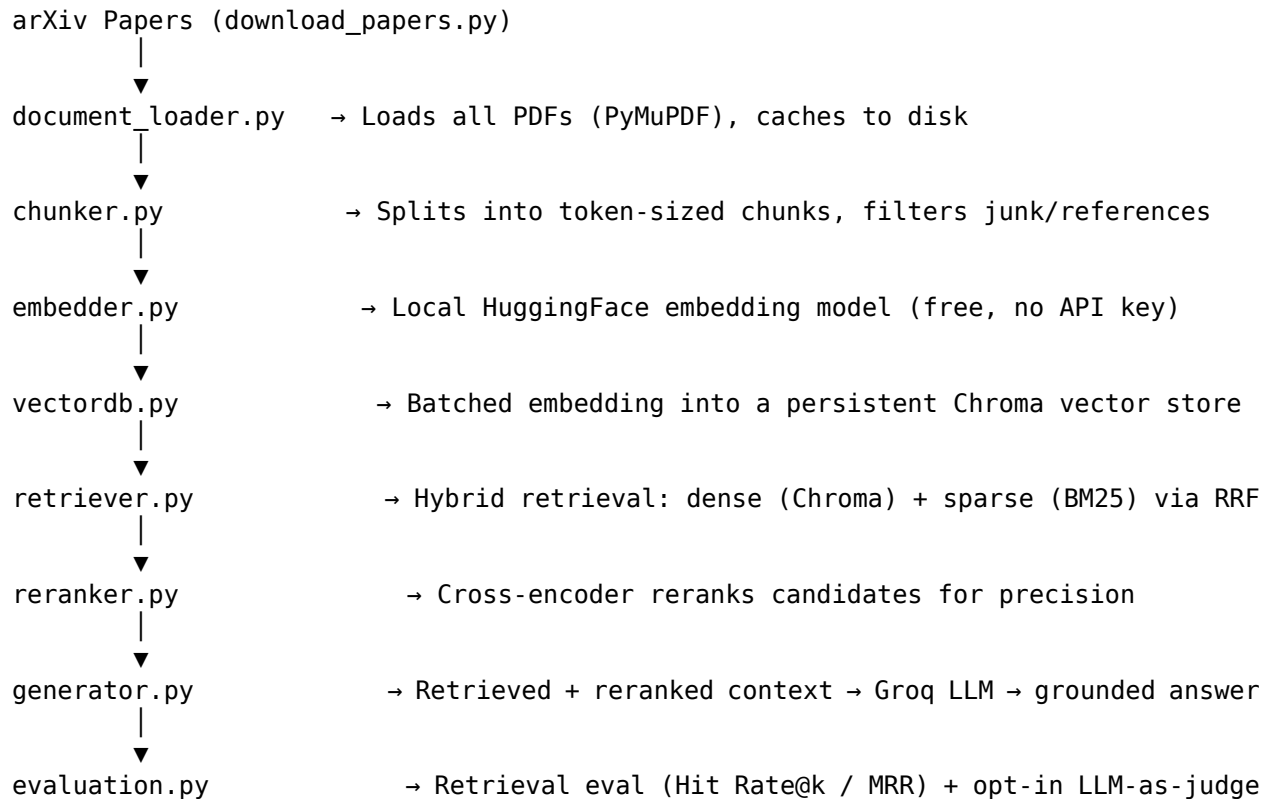
A Retrieval-Augmented Generation (RAG) research assistant that ingests a corpus of academic papers (arXiv, ~200 papers on LLM evaluation) and answers questions grounded in that corpus, using hybrid (dense + sparse) retrieval and cross-encoder reranking for high-precision results.

This project evolved from a single-PDF RAG prototype into a multi-document research assistant, built branch-by-branch on top of the original LearnRAG pipeline.

What This Project Does

1. Downloads a focused corpus of ~200 research papers from arXiv on a chosen topic (LLM evaluation).
2. Loads and parses all PDFs, with caching so this expensive step only runs once.
3. Splits papers into clean, filtered chunks (dropping junk/reference-list fragments).
4. Embeds chunks locally (free, no API cost) and stores them in a persistent Chroma vector database.
5. Retrieves relevant chunks using **hybrid search** — dense (semantic) + sparse (keyword/BM25) — fused via Reciprocal Rank Fusion.
6. Reranks the retrieved candidates with a cross-encoder model for final precision.
7. Generates a grounded answer using a Groq-hosted LLM, using only the reranked context.
8. Evaluates retrieval quality across dense-only, hybrid, and hybrid+reranked variants using Hit Rate@k and MRR on a 20-question test set.

Pipeline Overview



Project Structure

```
LearnRAG/
├── src/
│   └── learnrag/
│       ├── download_papers.py # Downloads ~200 arXiv papers on a chosen topic
│       ├── document_loader.py # Loads PDFs (PyMuPDF), with disk caching
│       ├── chunker.py # Recursive token-based chunking + junk filtering
│       ├── embedder.py # Local HuggingFace embedding model
│       ├── vectordb.py # Batched Chroma vector store build/load/search
│       ├── retriever.py # Hybrid retrieval (dense + sparse + RRF fusion)
│       ├── reranker.py # Cross-encoder reranking of retrieved candidates
│       ├── generator.py # Full RAG chain: retrieve → rerank → generate
│       ├── eval_dataset.py # 20-question evaluation test set
│       └── evaluation.py # Retrieval eval harness (Hit Rate@k / MRR)
├── docs/
│   └── research_papers/ # Downloaded arXiv PDFs (~200 papers)
├── cache/
│   └── documents.pkl # Cached parsed documents (avoids PDF re-parsing)
├── chroma_db/ # Persisted vector database (generated)
├── eval_results/ # Evaluation run outputs (generated, gitignored)
├── requirements.txt
├── .env # API keys (not committed)
├── .gitignore
└── README.md
```

Module Reference

`download_papers.py`

Uses the arxiv package to search and download a focused set of papers (e.g., ~200 papers matching “LLM evaluation” within cs.CL). Downloads each PDF via requests using each result’s `pdf_url` (the package’s older `download_pdf()` convenience method was removed in recent versions). Skips already-downloaded files so re-runs are safe.

`document_loader.py`

- `pdf_loader()` — loads a single PDF using PyMuPDFLoader (faster and more robust than pypdf for multi-column academic PDFs).
- `load_pdf_folder()` — loads every PDF in a folder, skipping and logging any that fail to parse (corrupt/scanned files).
- `get_documents()` — wraps folder loading with a pickle-based cache (`cache/documents.pkl`), so repeated runs/tests don’t re-parse ~200 PDFs every time. Use `force_reload=True` after adding/removing papers.

`chunker.py`

Splits documents using `RecursiveCharacterTextSplitter`, measured by **token count** (via `tiktoken`) rather than raw characters, for more predictable prompt sizing. Filters out: - Near-empty chunks (stray headers, page numbers) - Likely bibliography/reference-list fragments, detected heuristically (dense bracketed citation numbers, or repeated “Name Name,” author-list patterns)

Semantic chunking (embedding-based) was evaluated but dropped in favor of recursive splitting at this corpus scale — it makes an embedding call per sentence boundary, which is too slow across ~200 papers.

`embedder.py`

Loads a local HuggingFace embedding model (`sentence-transformers/all-MiniLM-L6-v2`). Runs entirely on-device — no API key or cost, though setting `HF_TOKEN` avoids rate-limit warnings on model download.

`vectordb.py`

- `build_vector_store()` — embeds chunks and writes them to a persistent Chroma collection **in batches** (important at ~5,500+ chunks, to avoid one huge blocking call).
- `load_vector_store()` — reloads the existing store without re-embedding.
- `similarity_search()` — plain dense similarity search (used for baseline comparison against hybrid search).

`retriever.py`

Implements **hybrid search**: - `build_bm25_retriever()` — sparse keyword retriever (BM25), built directly from chunk text, no embedding model needed. - `build_hybrid_retriever()` — combines the dense Chroma retriever and the BM25 retriever using `EnsembleRetriever`, which performs **weighted Reciprocal Rank Fusion (RRF)**: $\text{score} = \text{weight} / (\text{rank} + c)$, summed across retrievers and deduplicated by chunk content. - `hybrid_search()` — runs a query through the combined retriever and returns a ranked candidate set (recall-oriented: retrieves more candidates than are ultimately needed, e.g. top 10 from each retriever).

reranker.py

Implements the **precision** stage on top of retrieval: - Uses a cross-encoder model (cross-encoder/ms-marco-MiniLM-L-6-v2) that scores each (query, chunk) pair **jointly**, which is more accurate than comparing independently-computed embedding vectors or BM25 scores. - Too slow to run over the whole corpus, so it only reranks the candidate set produced by retriever.py (e.g., narrows ~15-20 candidates down to the top 3-5).

generator.py

Wires the full pipeline together using LangChain Expression Language (LCEL), with **lazy pipeline setup** (everything is built on first use, not at import — so evaluation.py can import the same functions without rebuilding): 1. Takes a user question. 2. Runs it through hybrid_search() (retrieval) then rerank() (precision) via retrieve_top_documents(). 3. Joins the final top chunks into a context string (retrieve_context()). 4. Formats context + question into a prompt instructing the model to answer **only** from the given context, and to explicitly say “I don’t know” if the context is insufficient. 5. Sends the prompt to a Groq-hosted LLM (openai/gpt-oss-120b) and parses the text response.

Helpers: - get_pipeline() — loads and caches the document → chunk → vector store → hybrid retriever stack. - build_rag_chain() — returns the LCEL chain that turns {"question", "context"} into an answer (retrieval left outside so callers can reuse one computed context, e.g. the eval harness). - generate(question) — full pipeline, returns (answer, source_paper_ids). - extract_sources(documents) — arXiv IDs of the papers behind a list of chunks. - Running python generator.py launches an **interactive chat**: type a question, get a grounded answer plus the source paper IDs; exit/quit/Ctrl-C to leave.

eval_dataset.py

EVAL_QUESTIONS — 20 questions, each paired with the arXiv ID(s) of the paper(s) a good retriever should surface (source-level relevance labels, a practical stand-in for expensive per-chunk labeling). Built partly by hand and partly mined from the corpus’s own paper titles/abstracts, so every expected source exists in docs/research_papers.

evaluation.py

The **evaluation harness** (see the Evaluation Harness section below). Compares three retrieval variants (dense-only, hybrid/RRF, hybrid+cross-encoder) with Hit Rate@k and MRR, prints a per-question breakdown, and saves JSON results to eval_results/. An LLM-as-judge faithfulness pass is available but **opt-in** (- -with- judge) to avoid burning API tokens by default.

Setup

1. Install dependencies

```
pip install -r requirements.txt
```

2. Set environment variables

Create a .env file in the project root:

```
GROQ_API_KEY=your_groq_api_key_here  
HF_TOKEN=your_huggingface_token_here
```

3. Download the research paper corpus

```
cd src/learnrag
python download_papers.py
```

4. Run the full pipeline

```
cd src/learnrag
python generator.py          # interactive chat (type a question, 'exit' to quit)
```

Note: the code uses CWD-relative paths (`./docs`, `./chroma_db`, `./cache`), so run the scripts from inside `src/learnrag/` (as above) — `python src/learnrag/generator.py` from the repo root will fail.

Why Hybrid Search + Reranking (Not Just Dense Retrieval)

Dense embeddings are excellent at matching *meaning*, but research queries often hinge on exact terms — benchmark names (MMLU, HELM), author names, specific metrics — that dense similarity alone can under-match. Sparse (BM25) search catches these exact-term matches; combining both via RRF improves recall over either alone. The cross-encoder reranker then adds a precision pass: it directly judges relevance of each candidate to the query, catching false positives (e.g., keyword-overlapping but topically irrelevant chunks) that rank fusion alone can let through.

Evaluation Harness

`evaluation.py` measures retrieval quality across the three variants used in the pipeline. Each variant retrieves from the **same deep candidate pool** (top 20) so the comparison is fair, then is scored at multiple cutoffs against the 20-question test set in `eval_dataset.py`.

Run it (default: retrieval only — no LLM calls, no API cost):

```
cd src/learnrag
python evaluation.py
```

Metrics: - **Hit Rate@k** — fraction of questions where at least one of the top-k retrieved chunks comes from an expected source paper. - **MRR** (Mean Reciprocal Rank) — average of $1/\text{rank}$ of the first expected source.

Results (20 questions, run 2026-08-25):

Variant	Hit@1	Hit@3	Hit@5	Hit@10	MRR
Dense-only (similarity_search)	65%	80%	90%	95%	0.746
Hybrid (dense + BM25, RRF)	70%	85%	85%	90%	0.778
Hybrid + Cross-Encoder	75%	80%	80%	80%	0.767

- The cross-encoder variant beats everything at Hit@1 (75%) but returns only the production top_k=3 chunks, so its Hit Rate plateaus after k=3 — that is by design, not a bug.
- RRF fusion improves top precision over dense alone (70% vs 65% Hit@1, higher MRR), but dense alone has the deepest recall (95% at @10).
- Three questions (Q1 2502.10709, Q13 2407.21072, Q18 2403.07872) are missed by **all** variants — good candidates to inspect when improving retrieval or question/label quality.

Every run also saves a full breakdown (per-question hits, config) as JSON to `eval_results/eval_<timestamp>` (gitignored), so results can be tracked across changes.

LLM-as-judge (opt-in, future enhancement)

An optional pass scores **generation faithfulness** — whether the answer is actually grounded in the retrieved context (1-5) — via a ChatGroq judge. It is disabled by default because it makes LLM calls per question (answer generation + judge) and the Groq free tier caps gpt-oss-120b at 8000 tokens/minute, making it slow and rate-limit prone:

```
python evaluation.py --with-judge [--judge-runs N]    # N judge passes/question, default 1
```

Known Limitations / Notes

- Table and figure-caption content sometimes extracts as jumbled text fragments (PDF parsers don't understand table structure) — occasionally surfaces as a noisy retrieval result.
- A small number of downloaded papers (~3 out of 200) may fail to load (corrupted download or scanned/image-only PDF); these are logged and skipped rather than crashing the pipeline.
- Groq periodically deprecates model IDs (e.g., llama-3.3-70b-versatile → openai/gpt-oss-120b); check <https://console.groq.com/docs/models> if you hit a `model_not_found` error.
- The Groq free tier's 8000 tokens/minute cap on openai/gpt-oss-120b makes the LLM-as-judge faithfulness pass slow and rate-limit prone (hence it is opt-in). Generation in the interactive chat is unaffected for normal single questions.

Possible Next Steps

- **Improve retrieval on the missed eval questions** — Q1, Q13, Q18 are missed by all three variants; a good next experiment (e.g., tuning RRF weights/c, chunk size, or query expansion).
- **Source-aware citations** — surface which paper(s) each answer draws from directly in the chat response (metadata is already tracked per chunk and the REPL already prints source IDs).
- **LLM-as-judge hardening** — when a higher Groq tier (or another provider) is available, enable `--with-judge` by default and add answer-relevance scoring alongside faithfulness.
- **Deploy with Docker** — containerize the app; Chroma with a mounted volume is sufficient for a single-host deployment, Pinecone becomes worth considering only for serverless/multi-replica deployments.
- **UI layer** — wrap the pipeline in a simple Streamlit or Gradio front end.

Tech Stack

Component	Tool
Paper acquisition	arxiv package (arXiv API)
PDF Loading	PyMuPDFLoader (langchain-community)
Text Splitting	RecursiveCharacterTextSplitter (token-based, via tiktoken)

Component	Tool
Embeddings	HuggingFace sentence-transformers/all-MiniLM-L6-v2 (local, free)
Vector Database	ChromaDB (batched writes)
Sparse Retrieval	BM25 (rank_bm25 via BM25Retriever)
Hybrid Fusion	EnsembleRetriever (Reciprocal Rank Fusion)
Reranking	cross-encoder/ms-marco-MiniLM-L-6-v2 (local, free)
LLM Inference	Groq (openai/gpt-oss-120b)
Orchestration	LangChain Expression Language (LCEL)
Evaluation	Retrieval metrics (Hit Rate@k, MRR) + opt-in LLM-as-judge faithfulness